

Cryptography

Breaking the Vigenère cipher

Breaking the Vigenère cipher

- Needed for programming assignment 1

The Vigenère cipher

- The key is a string of letters
- To encrypt, shift each character in the plaintext by the amount dictated by the next character of the key
 - Wrap around in the key as needed
- Decryption just reverses the process

```
tellhimaboutme  
cafecafecafeca  
veqnpjiredozxoe
```

Variant Vigenère cipher

- Easier to work with ASCII plaintext and hex ciphertext
 - Easier to implement
 - Easier to use (plaintext not limited to lowercase characters)
- Easier to work with byte-wise XOR rather than modular addition

Variant Vigenère cipher

- The key is a string of bytes
- The plaintext is a string of ASCII characters
- To encrypt, XOR each character in the plaintext with the next character of the key
 - Wrap around in the key as needed
- Decryption just reverses the process

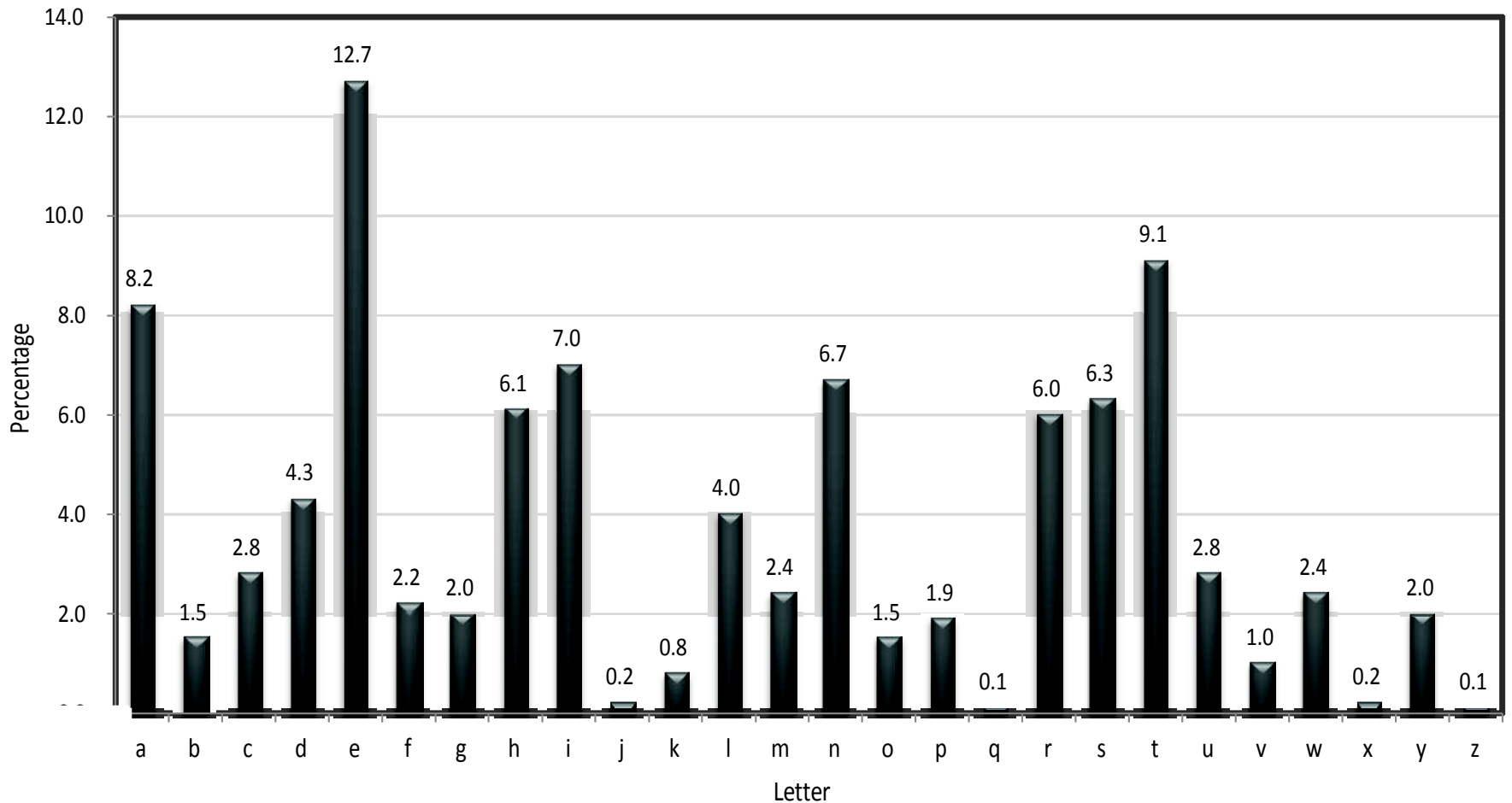
Example

- Say plaintext is “Hello!” and key is 0xA1 2F
- “Hello!” = 0x48 65 6C 6C 6F 21
- XOR with 0xA1 2F A1 2F A1 2F
- $0x48 \oplus 0xA1$
 - $0100\ 1000 \oplus 1010\ 0001 = 1110\ 1001 = 0xE9$
- Ciphertext: 0xE9 4A CD 43 CE 0E

Attacking the (variant) Vigenère cipher

- Two steps:
 - Determine the key length
 - Determine each byte of the key

Known plaintext letter frequencies



Determining the key length

- Let p_i (for $0 \leq i \leq 255$) be the frequency of byte i in plaintext (assuming English text)
 - I.e., $p_i = 0$ for $i < 32$ or $i > 127$
 - I.e., p_{97} = frequency of 'a'
 - The distribution is far from uniform
- If the key length is N , then every N^{th} character of the plaintext is encrypted using the same “shift”
 - If we take every N^{th} character and calculate frequencies, we should get the p_i 's in permuted order
 - If we take every M^{th} character (M not a multiple of N) and calculate frequencies, we should get something close to uniform

Determining the key length

- How to distinguish these two?
- For some candidate distribution $q_0, \dots, q_{255}$, compute $\sum q_i^2$
 - If close to uniform, $\sum q_i^2 \approx 256 \cdot (1/256)^2 = 1/256$
 - If a permutation of p_i , then $\sum q_i^2 \approx \sum p_i^2$
 - Could compute $\sum p_i^2$ (but somewhat difficult)
 - Key point: will be much larger than $1/256$
- Try all possibilities for the key length, compute $\sum q_i^2$, and look for maximum value

Determining the i^{th} byte of the key

- Assume the key length N is known
- Look at every N^{th} character of the ciphertext, starting with the i^{th} character
 - Call this the i^{th} ciphertext “stream”
 - Note that all bytes in this stream were generated by XORing plaintext with the same byte of the key
- Try decrypting the stream using every possible byte value B
 - Get a candidate plaintext stream for each value

Determining the i^{th} byte of the key

- When the guess B is correct:
 - All bytes in the plaintext stream will be between 32 and 127
 - Frequencies of lowercase letters (as a fraction of all lowercase letters) should be close to known English-letter frequencies
 - Tabulate $q_a, \dots, q_z$
 - Should find $\sum q_i p_i \approx \sum p_i^2 \approx 0.065$
 - In practice, take B that maximizes $\sum q_i p_i$, subject to caveat above (and possibly others)

Attack time?

- Say the key length is between 1 and L
- Determining the key length: $\approx 256 L$
- Determining all bytes of the key: $\approx 256^2 L$
- Brute-force key search: $\approx 256^L$

The attack in practice

- Attacks get more reliable as the ciphertext length grows larger
- Attacks still work for short(er) ciphertexts, but more “tweaking” and manual involvement is needed
 - You should expect to have to do this for the HW!

Programming assignment

- Use discussion boards for help
- Good luck!